ARGUS Quick Start

Every command, labelled PI or LAPTOP

ARGUS quick start

Every block says which machine it runs on. **PI** is a Raspberry Pi Connect shell; **LAPTOP** is PowerShell on Windows.

---

1 · PI — start the node

```
cd ~/argus
git pull
./scripts/run_node.sh
```

It prints the address to open and then stays running. Leave the window alone.

Want	Command
Classify subjects that hold still	`./scripts/run_node.sh --every-frame`
Bigger detector	`./scripts/run_node.sh --model yolov8s`
Run against a recording, no camera	`./scripts/run_node.sh --video media/vtest.avi`
Add the dangerous-person stage	`./scripts/run_node.sh --threat`

First run only, to get the fast CPU path:

```
python scripts/export_cpu_models.py --format ncnn
```

---

2 · Watch it

Open `http://<pi-address>:8000/dashboard` in any browser that can reach the Pi. Live feed with boxes, detection FPS every second, what is in frame, and events with the still saved for each.

Or from a shell:

PI

```
cd ~/argus && ./scripts/node_status.sh
```

One line per second:

```
fps= 9.9 ms=   27 cap= 10.0 | tracks= 2 events=  14 motion=  3 stills=  14 | cpu-ncnn yolov8n
gate=True rate=0.98 | fl-r2
```

`tracks` rises when something is in frame; `events` rises once it leaves.

---

3 · PI — check what the camera sees

```
cd ~/argus && source .venv/bin/activate
python scripts/check_classes.py --camera --save /tmp/seen.jpg
```

Prints the ARGUS classes and the raw COCO labels behind them, and writes an annotated copy you can open. Point it at a file instead to test without the camera:

```
python scripts/check_classes.py media/vtest.avi
```

The four classes: person · vehicle (bicycle, car, motorcycle, bus, train, truck) · animal (bird, cat, dog, horse, sheep, cow) · package (backpack, handbag, suitcase). COCO has no parcel class, so a cardboard box detects as nothing — use a backpack.

---

4 · PI — a federated round

Node running in shell 1, then:

```
cd ~/argus && ./scripts/fl_trial.sh
```

It starts the aggregator, seeds samples, trains locally, joins the round, and reports the gate's decision. Watch `detect_fps` while it runs — training is a separate process on one core, so detection keeps going.

`model_version` moves to `fl-r1`, `fl-r2` … only when the gate **accepts** the averaged model. A rejection means it scored worse on this node's own held-out samples, and the node kept the model it had. That is the gate working.

> The seeded samples are synthetic and never appear in the event list. Say so when presenting: the pipeline captures and trains on real events, demonstrated alongside a corpus large enough to train on.

4b · The central server, and the schedule

The laptop is where the cameras send what they learned. Weights only — no video, no images.

LAPTOP — start it

```
cd $HOME\argus\argus
.\scripts\run_central_server.ps1
```

It prints the two lines to set on the Pi, keeps every version of the combined model in `central_server/checkpoints`, and stays up between sessions so a scheduled camera finds it days later.

Watch	URL
Rounds, loss, accuracy	`http://localhost:8090/api/training/summary`
Cameras and their scores	`http://localhost:8090/api/nodes`
Prometheus	`http://localhost:8090/metrics`

PI — join on a schedule

```
export FL_ENABLED=true
export FL_SERVER_URL=192.168.1.50:8080
export FL_CENTRAL_API=http://192.168.1.50:8090
./scripts/run_node.sh
```

Rounds then run by themselves: every 14 days at 2am, only while nothing is moving, catching up if the Pi was off across the due date. Check the schedule any time:

```
curl -s localhost:8000/api/federated/status | python -m json.tool
```

PI — turn saved video into training data

```
python scripts/ingest_video.py media/events/some_clip.mp4 --label routine_person
```

Labels: `routine_person`, `routine_vehicle`, `routine_animal`, `routine_package`, `anomaly`. Stop the node first — both write to the same store.

---

5 · LAPTOP — the product dashboard (optional)

```
cd $HOME\argus\argus
.\scripts\run_dashboard.ps1 -Node 10.0.0.140
```

Then open `http://localhost:3000` (or `:3001`) and log in as `admin`.

Set the password on the **PI**:

```
python scripts/set_password.py --user admin
```

If it says SYSTEM OFFLINE, your laptop cannot reach the Pi, or the port Next.js chose is missing from `CORS_ORIGINS`. The node's own page at `http://<pi>:8000/dashboard` has no such dependency.

If it breaks

Symptom	Fix
Only `person` detected	Subject is holding still → `--every-frame`; or it is too small in frame
Nothing detected at all	`python scripts/check_classes.py --camera` — separates model from scene
`camera_error` in status	`python -c "from picamera2 import Picamera2"`; if it mentions numpy → `pip install --ignore-installed "numpy>=2.2,<3"`
`ImportError: ... Bad message`	A system library is **corrupt**, not missing. See "When the card goes bad" below
People look blue	`CAMERA_SWAP_RB=true ./scripts/run_node.sh`
Round returns 409	Fewer than 200 labelled samples — let `fl_trial.sh` seed them
Dashboard offline, curl works	Wrong port in `CORS_ORIGINS` (Next.js often uses 3001)
`backend: cpu` on the Pi	`python scripts/export_cpu_models.py --format ncnn`

When the card goes bad

`Bad message` in an `ImportError` is **EBADMSG** — ext4 refusing to hand over a file whose checksum no longer matches. The file is corrupt, not missing, and reinstalling with `apt` may not even run: `apt` is C++, so a damaged `libstdc++` takes it down too. An unclean shutdown is the usual cause.

`dpkg` records an md5 for every file it installed, so the whole system can be verified offline. **PI**

```
cd / && cat /var/lib/dpkg/info/*.md5sums | md5sum -c 2>/dev/null | grep -v ': OK$' | sed 's/:
FAILED.*//' | sed 's|^|/|' > /tmp/bad.txt
wc -l < /tmp/bad.txt
```

Map those files back to their packages and reinstall the lot. The `grep -v '^diversion'` matters — `dpkg -S` emits diversion lines that are not package names:

```
xargs -a /tmp/bad.txt dpkg -S 2>/dev/null | grep -v '^diversion' | cut -d: -f1 | tr ',' '\n' | tr -d
' ' | grep -v '^$' | sort -u > /tmp/badpkgs.txt
sudo apt install --reinstall -y $(tr '\n' ' ' < /tmp/badpkgs.txt)
```

If `apt` itself will not start, repair `libstdc++` first with `dpkg`, which is plain C. Take the version from `dpkg -l libstdc++6`:

```
cd /tmp
U=http://deb.debian.org/debian/pool/main/g/gcc-14
curl -fLO $U/libstdc++6_14.2.0-19_arm64.deb
sudo dpkg -i /tmp/libstdc++6_14.2.0-19_arm64.deb && sudo ldconfig
```

Re-run the scan afterwards. Zero failures means every tracked file matches Debian's published hash. Reboot if the kernel, `systemd`, `kmod` or the Hailo driver were among them, then confirm with `hailortcli fw-control identify`.

A file that fails again after being rewritten means the card is bad — reimage, and do not reuse it. Check `dmesg | grep -iE 'mmc0|I/O error'`: a silent failure with no I/O errors logged is typical of a counterfeit or worn card.